

МИНОБРНАУКИ РОССИИ
Санкт-Петербургский государственный
электротехнический университет
«ЛЭТИ» им. В.И. Ульянова (Ленина)
Кафедра МО ЭВМ

Отчет
по лабораторной работе №3
по дисциплине «Введение в машинное обучение»
Тема: Классификация.

Студентка гр. 3384

Копасова К. А.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Цель работы: Изучить классификацию данных.

Задание: Вариант 4 - lab3_4.csv.

1. Загрузка данных

1.1. Загрузите данные вашего варианта. Учтите, что метки классов могут быть текстом.

1.2. Визуализируйте данные при помощи диаграммы рассеяния с выделением различных классов ней.

1.3. Оцените сбалансированность классов.

1.4. Проведите предобработку данных при необходимости.

1.5. Разделите выборку на обучающую и тестовую. На обучающей проводите обучение модели, на тестовой расчет значений метрик.

2. kNN

2.1. Проведите классификацию методом k ближайших соседей, подобрав параметры: количество соседей, необходимость взвешивания. Постройте графики зависимости точности (Accuracy) в зависимости от количества соседей (по 1-й линии для взвешенного и не взвешенного метода)

2.2. Постройте изображение с границами принятия решения ответив на них точки классов разным цветом. Сделайте выводы о качестве классификации на основе полученного изображения.

2.3. Постройте таблицу ошибок для полученных результатов. Сделайте выводы о классификации на основе таблицы ошибок.

2.4. Рассчитайте значения Precision, Recall, F1 для полученных результатов. Сопоставьте с таблицей ошибок.

2.5. Постройте изображение ROC-кривой и рассчитайте значение AUC для полученных результатов. Сделайте выводы о классификации по полученному изображению и значению AUC.

3. Логистическая регрессия

3.1. Проведите классификацию методом логистической регрессии при различных параметрах: без регуляризации, с l1 регуляризацией, с l2

регуляризацией. Постройте столбчатую диаграмму зависимости точности (Accuracy) от наличия регуляризации. Дальнейшие пункты 3.* выполняйте для лучшего параметра.

3.2. Постройте изображение с границами принятия решения ответив на них точки классов разным цветом. Сделайте выводы о качестве классификации на основе полученного изображения.

3.3. Постройте таблицу ошибок для полученных результатов. Сделайте выводы о классификации на основе таблицы ошибок.

3.4. Рассчитайте значения Precision, Recall, F1 для полученных результатов. Сопоставьте с таблицей ошибок.

3.5. Постройте изображение ROC-кривой и рассчитайте значение AUC для полученных результатов. Сделайте выводы о классификации по полученному изображению и значению AUC.

4. Метод опорных векторов

4.1. Проведите классификацию методом опорных векторов при различных параметрах ядра: “linear”, “poly” (нужно выбрать степень), “rbf”. Постройте столбчатую диаграмму зависимости точности (Accuracy) от вида ядра. Дальнейшие пункты 4.* выполняйте для лучшего параметра.

4.2. Постройте изображение с границами принятия решения ответив на них точки классов разным цветом. Сделайте выводы о качестве классификации на основе полученного изображения.

4.3. Постройте таблицу ошибок для полученных результатов. Сделайте выводы о классификации на основе таблицы ошибок.

4.4. Рассчитайте значения Precision, Recall, F1 для полученных результатов. Сопоставьте с таблицей ошибок.

4.5. Постройте изображение ROC-кривой и рассчитайте значение AUC для полученных результатов. Сделайте выводы о классификации по полученному изображению и значению AUC.

5. Решающие деревья

5.1. Проведите классификацию используя решающие деревья, подобрав параметры при которых получается лучшее обобщение (максимальная глубина/максимальное количество листьев/метрика загрязнения и т.д.).

5.2. Постройте изображение с границами принятия решения ответив на них точки классов разным цветом. Сделайте выводы о качестве классификации на основе полученного изображения.

5.3. Постройте таблицу ошибок для полученных результатов. Сделайте выводы о классификации на основе таблицы ошибок.

5.4. Рассчитайте значения Precision, Recall, F1 для полученных результатов. Сопоставьте с таблицей ошибок.

5.5. Постройте изображение ROC-кривой и рассчитайте значение AUC для полученных результатов. Сделайте выводы о классификации по полученному изображению и значению AUC.

5.6. Визуализируйте полученное дерево решений. Сделайте выводы о правилах в узлах дерева. Сопоставьте их с полученными границами принятия решений.

6. Выбор классификатора

6.1. Постройте таблицу с метриками Precision, Recall, AUC для полученных результатов каждым классификатором.

6.2. Сделайте выводы о том, какие классификаторы лучше всего подходят для вашего набора данных и почему.

Выполнение работы

1. Загрузка данных

1.1. Загрузка данных.

Загрузим данные из файла lab3_4.csv через функцию read_csv() библиотеки pandas.

```
import pandas as pd
data = pd.read_csv('lab3_4.csv')
print(data.head())
```

Результат работы показан на рис. 1.1.1.

	Unnamed: 0	Experience	KPI	Status
0	0	18.7	10.22	Fire
1	1	12.0	15.11	Fire
2	2	10.5	21.74	Fire
3	3	8.1	10.70	Fire
4	4	23.5	18.74	Fire

Рисунок 1.1.1 - Демонстрация набора данных.

Набор данных содержит следующие столбцы:

- 1) Unnamed: 0 - индекс строки;
- 2) Experience - числовая характеристика, стаж сотрудника;
- 3) KPI - числовая характеристика, показатель эффективности сотрудника;
- 4) Status - категориальная метка, обозначает принадлежность сотрудника к какому-то классу.

Status является целевой переменной, а признаки Experience и KPI будут использоваться для определения принадлежности сотрудника к соответствующему классу.

Закодируем столбец Status с помощью LabelEncoder, чтобы корректно использовать его в дальнейшем.

```
from sklearn.preprocessing import LabelEncoder

label_encoder = LabelEncoder()
data['Status'] = label_encoder.fit_transform(data['Status'])

print(data.head())
```

Результат работы показан на рис. 1.1.2.

	Unnamed: 0	Experience	KPI	Status
0	0	18.7	10.22	0
1	1	12.0	15.11	0
2	2	10.5	21.74	0
3	3	8.1	10.70	0
4	4	23.5	18.74	0

Рисунок 1.1.2 - Демонстрация кодированного набора данных.

Каждый класс Status после энкодера представлен не строковым значением, а числовым кодом.

1.2. Визуализация данных при помощи диаграммы рассеяния с выделением различных классов ней.

Построим диаграмму рассеяния с выделением различных классов.

```
import matplotlib.pyplot as plt

classes = data['Status'].unique()

for status in classes:
    subset = data[data['Status'] == status]
    plt.scatter(subset['Experience'], subset['KPI'], label=status)

plt.xlabel('Experience')
plt.ylabel('KPI')
plt.title('Диаграмма рассеяния признаков')
plt.legend()
plt.show()
```

Результат работы показан на рис. 1.2.

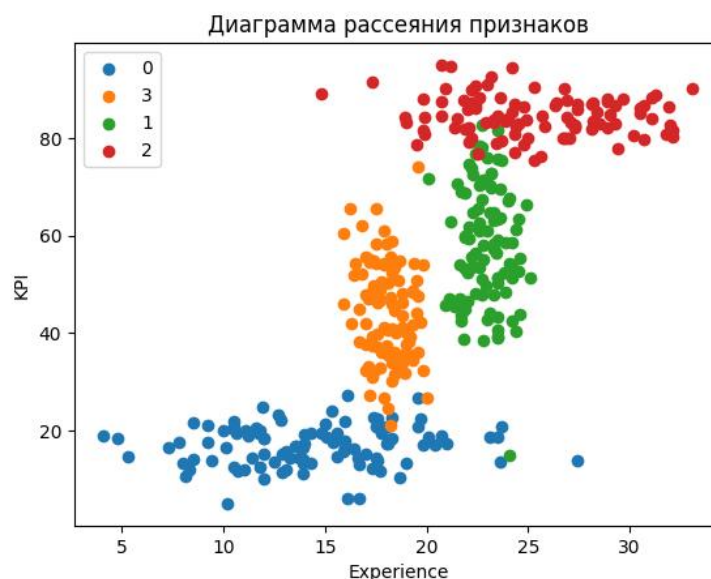


Рисунок 1.2 - Диаграмма рассеяния признаков.

На диаграмме наблюдается разделение данных на четыре класса: 0 класс (синие точки) - имеет низкий KPI (10-25) и широкий диапазон стажа (5-30), 1 класс (зеленые точки) - имеет средне-высокий KPI (40-80) и средний диапазон стажа (20-25), 2 класс (красные точки) - имеет высокий KPI (80-90) и высокий стаж (20-30) и 3 класс (оранжевые точки) - имеет средний KPI (25-65) и низкий стаж (15-20).

Классы визуально разделены, но имеют зоны перекрытия:

- 1) Классы 1 и 2 частично пересекаются в области Experience ~ 22-25, KPI ~ 70-80;
- 2) Классы 0 и 3 имеют небольшое пересечение при Experience ~ 15-18;
- 3) Класс 0 наиболее обособлен от других.

Также у всех классов наблюдаются небольшие выбросы.

1.3. Оценка сбалансированности классов.

Для оценки сбалансированности классов рассмотрим распределением количества объектов по каждому классу.

```
class_counts = data['Status'].value_counts()

plt.figure(figsize=(6,4))
plt.bar(class_counts.index, class_counts.values)
plt.xlabel('class')
plt.ylabel('count')
plt.title('Распределение объектов по классам')
plt.show()
```

Результат работы показан на рис. 1.3.



Рисунок 1.3 - Распределение объектов по классам.

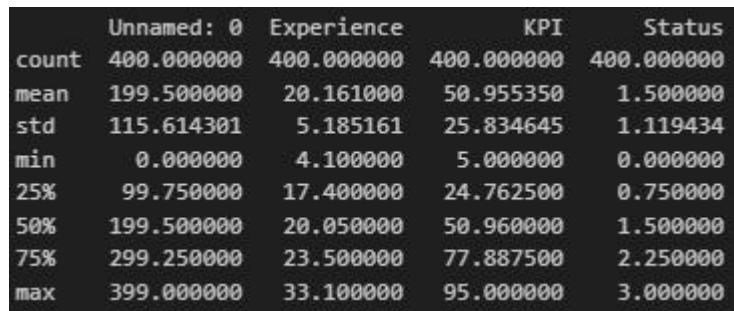
Количество объектов во всех классах равно 100, следовательно, набор данных можно считать сбалансированным.

1.4. Предобработка данных.

Проверим статистические характеристики признаков через метод `describe()`.

```
print(data.describe())
```

Результат работы показан на рис. 1.4.1.



	Unnamed: 0	Experience	KPI	Status
count	400.000000	400.000000	400.000000	400.000000
mean	199.500000	20.161000	50.955350	1.500000
std	115.614301	5.185161	25.834645	1.119434
min	0.000000	4.100000	5.000000	0.000000
25%	99.750000	17.400000	24.762500	0.750000
50%	199.500000	20.050000	50.960000	1.500000
75%	299.250000	23.500000	77.887500	2.250000
max	399.000000	33.100000	95.000000	3.000000

Рисунок 1.4.1 - Статистические характеристики признаков.

По полученным статистическим характеристикам можно сделать следующие выводы:

- 1) в наборе данных содержится 400 наблюдений и пропущенные значения в признаках отсутствуют;
- 2) признак Experience принимает значения в диапазоне примерно от 4 до 33 лет;
- 3) признак KPI изменяется в диапазоне от 5 до 95, что соответствует возможному диапазону значений показателя эффективности;
- 4) значения целевой переменной Status находятся в диапазоне от 0 до 3, что соответствует четырем различным классам.

Данные не содержат пропусков и некорректных значений. Но признаки Experience и KPI имеют разный масштаб изменения - поэтому для корректности стандартизируем их. Для выбора наиболее подходящего метода масштабирования нужно проверить выбросы данных через `boxplot`.

```
print(plt.subplot(1,2,1))
plt.boxplot(data['Experience'])
plt.title('Experience')

plt.subplot(1,2,2)
plt.boxplot(data['KPI'])
plt.title('KPI')
```



```
plt.tight_layout()
plt.show()
```

Результат работы показан на рис. 1.4.2.

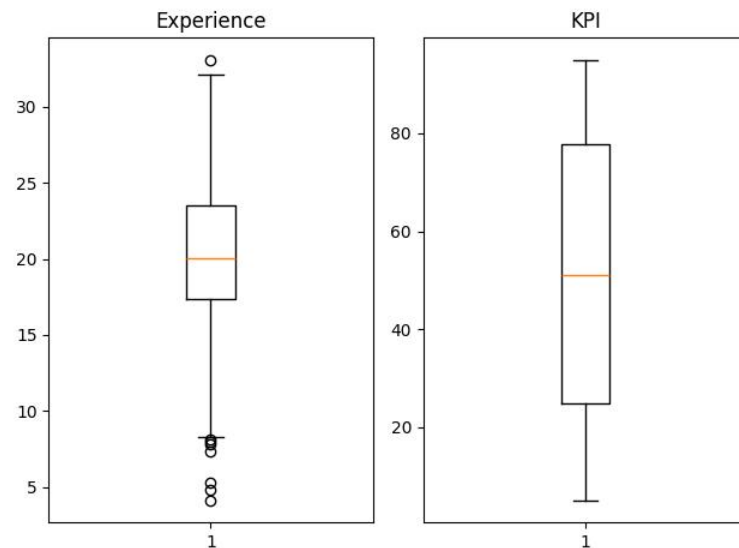


Рисунок 1.4.2 - Проверка признаков на выбросы.

По полученным диаграммам видим, что Experience имеет достаточно много выбросов - они могут оказывать влияние на работу алгоритмов машинного обучения, основанных на расстояниях между объектами. Поэтому для масштабирования признаков будем использовать устойчивый к выбросам метод - RobustScaler.

```
from sklearn.preprocessing import RobustScaler

rb_scaler = RobustScaler()
X = data[['Experience', 'KPI']]
X_scaled = rb_scaler.fit_transform(X)

print(X_scaled[:5])
```

Результат работы показан на рис. 1.4.3.

```
[[-0.22131148 -0.76687059]
 [-1.31967213 -0.67482353]
 [-1.56557377 -0.55002353]
 [-1.95901639 -0.75783529]
 [ 0.56557377 -0.60649412]]
```

Рисунок 1.4.3 - Предобработанные данные.

После RobustScaler признаки центрированы вокруг 0, масштабирование выполнено с использованием межквартильного размаха (IQR), что делает метод устойчивым к выбросам в признаке Experience.

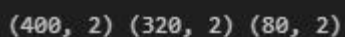
1.5. Разделение выборки на обучающую и тестовую.

Разделим данные на обучающую (80% данных) и тестовую (остальные 20%) выборки с помощью функции `train_test_split()` библиотеки `sklearn`. Через параметр `stratify` сохраняем соотношение классов в `test` и `train`, как в исходной выборке.

```
from sklearn.model_selection import train_test_split

y = data['Status']
X_train_scaled, X_test_scaled, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.2, random_state=41)
print(X.shape, X_train_scaled.shape, X_test_scaled.shape)
```

Результат работы показан на рис. 1.5.



(400, 2) (320, 2) (80, 2)

Рисунок 1.5 - Размерность всех данных и полученных выборок.

Данные разделены на обучающую (320 объектов - 80%) и тестовую (80 объектов - 20%) выборки.

2. kNN

2.1. Классификация методом k ближайших соседей с подбором параметров (количество соседей, необходимость взвешивания).

Метод k ближайших соседей - это алгоритм классификации, который относит объект к тому классу, который наиболее распространен среди его k ближайших соседей в пространстве признаков.

Для этого метода нужно подобрать оптимальное количество соседей (параметр `k`) и необходимость взвешивания (параметр `weights`). `Weights` определяет, будут ли все соседи иметь одинаковое влияние (`uniform`) или вес будет обратно пропорционален расстоянию (`distance`). Для подбора параметров был проведен перебор `k` от 1 до 30 для обоих типов взвешивания. Точность моделей оценивалась на тестовой выборке с помощью метрики `Accuracy`.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

k_range = range(1, 31)
acc_uniform = []
acc_distance = []
```

```

for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k, weights='uniform')
    knn.fit(X_train_scaled, y_train)
    acc_uniform.append(accuracy_score(y_test, knn.predict(X_test_scaled)))

    knn = KNeighborsClassifier(n_neighbors=k, weights='distance')
    knn.fit(X_train_scaled, y_train)
    acc_distance.append(accuracy_score(y_test, knn.predict(X_test_scaled)))

plt.plot(k_range, acc_uniform, marker='o', label='uniform')
plt.plot(k_range, acc_distance, marker='s', label='distance')
plt.xlabel('k (количество соседей)')
plt.ylabel('Accuracy')
plt.title('Зависимость точности от k')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

print(f"uniform:                                k={np.argmax(acc_uniform)+1},
Accuracy={max(acc_uniform):.4f}")
print(f"distance:                               k={np.argmax(acc_distance)+1},
Accuracy={max(acc_distance):.4f}")

```

Результат работы показан на рис. 2.1.1 и рис. 2.1.2.

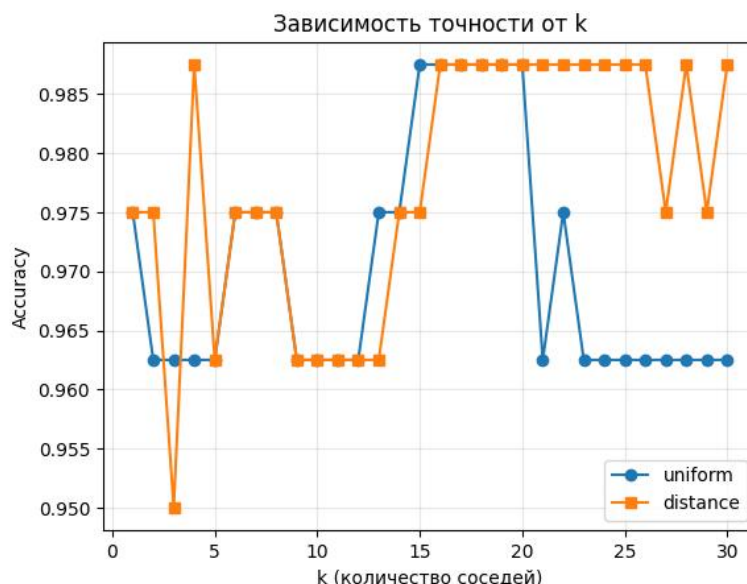


Рисунок 2.1.1 - Зависимость точности от k для методов uniform и distance.

```

uniform: k=15, Accuracy=0.9875
distance: k=4, Accuracy=0.9875

```

Рисунок 2.1.2 - Максимальные значения Accuracy для методов uniform и distance.

По графику заметно, что при малых k (1-5) точность нестабильна из-за чувствительности к отдельным объектам. При k>15 метод uniform теряет

точность из-за излишнего обобщения, в то время как distance сохраняет высокую точность благодаря учету расстояний.

Максимальная точность 0.9875 достигнута обоими методами но при разных k: uniform - k = 15, distance - k = 4.

Для дальнейшей работы выбран метод distance с k = 4, так как он обеспечивает ту же точность при меньшей вычислительной сложности и лучше учитывает близость объектов (ближние соседи влияют сильнее).

2.2. Построение изображения с границами принятия решения.

Построим границы принятия решений для kNN с помощью класса DecisionBoudaryDisplay из библиотеки sklearn.

```
from sklearn.inspection import DecisionBoundaryDisplay

knn_best = KNeighborsClassifier(n_neighbors=4, weights='distance')
knn_best.fit(X_train_scaled, y_train)

display = DecisionBoundaryDisplay.from_estimator(
    knn_best,
    X_scaled,
    response_method='predict',
    xlabel='Experience (масштабированный)',
    ylabel='KPI (масштабированный)',
    grid_resolution=200,
    alpha=0.7,
    cmap='viridis'
)

scatter = plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y, cmap='viridis',
edgecolors='k', s=50, linewidth=0.5)
plt.legend(*scatter.legend_elements(), title="Классы", loc='upper right')
plt.title('Границы принятия решений kNN (k=4, weights=distance)')
plt.grid(False)

plt.show()
```

Результат работы показан на рис. 2.2.

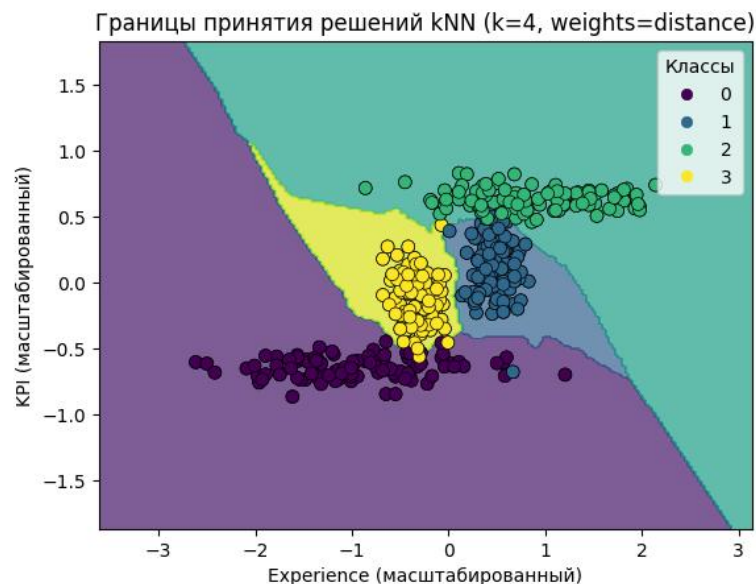


Рисунок 2.2 - Границы принятия решений kNN.

Границы между классами имеют нелинейный ступенчатый характер, что характерно для метода kNN. Алгоритм определяет класс каждого объекта по его ближайшим соседям. Класс 0 хорошо отделен в нижней части графика (низкий KPI), класс 3 находится слева (низкий Experience) - оба имеют четкие границы. Классы 1 и 2 частично пересекаются в центре графика, что уже наблюдалось на диаграмме рассеяния в п. 1.2. Большинство точек находятся в областях своего класса и $\text{Assurasy} = 0.9875$ - это говорит о хорошем качестве классификации. Некоторые точки лежат на границах или в чужих областях - это ошибки модели. Небольшие «островки» других классов показывают, что в данных есть пограничные объекты.

2.3. Построение таблицы ошибок для полученных результатов. Выводы о классификации на основе таблицы ошибок.

Построим таблицу ошибок kNN через класс `ConfusionMatrixDisplay` из библиотеки `sklearn`.

```
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_estimator(
    knn_best,
    X_test_scaled,
    y_test,
    display_labels=['0', '1', '2', '3'],
    cmap='Blues',
```

```

values_format='d'
)
plt.title('Матрица ошибок kNN (k=4, distance)')
plt.grid(False)
plt.show()

```

Результат работы показан на рис. 2.3.

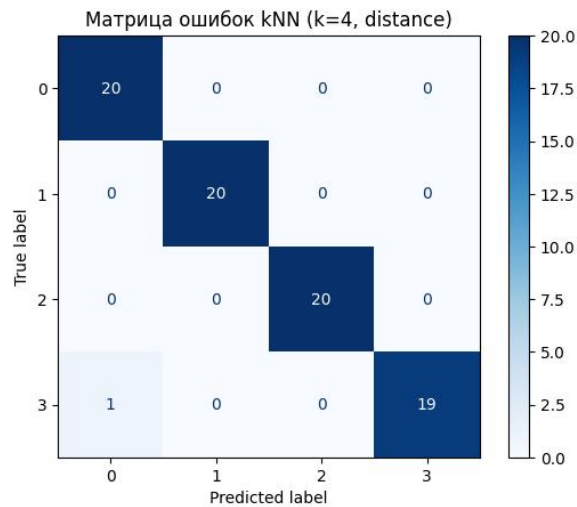


Рисунок 2.3 - Матрица ошибок kNN.

На матрице ошибок диагональные ячейки показывают количество правильно классифицированных объектов, а остальные - ошибки модели. Из 80 объектов тестовой выборки правильно предсказано 79, что соответствует точности 0.9875.

Основная ошибка (1 объект) произошла при классификации класса 3: один объект был предсказан как класс 0. Это может быть связано с тем, что оба класса имеют схожие значения признаков (низкий KPI у класса 0 и низкий Experience у класса 3), и в пограничной области модель ошиблась.

Остальные классы (0, 1, 2) классифицированы без ошибок: все 20 объектов каждого класса предсказаны верно.

2.4. Расчет значений Precision, Recall, F1 для полученных результатов. Сопоставление с таблицей ошибок.

Для оценки качества классификации рассчитаем метрики Precision (точность), Recall (полнота) и F1-мера (гармоническое среднее) для каждого класса. Precision показывает, какая доля объектов, предсказанных как данный класс, действительно принадлежит ему. Recall показывает, какая доля объектов

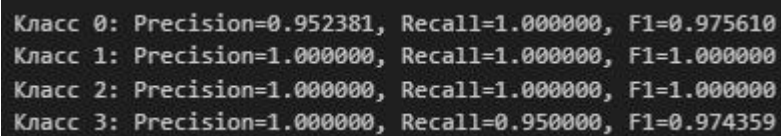
данного класса найдена моделью. F1-мера объединяет обе метрики в одну, являясь их гармоническим средним.

```
from sklearn.metrics import precision_recall_fscore_support

y_pred_knn = knn_best.predict(X_test_scaled)

precision, recall, f1, support = precision_recall_fscore_support(y_test,
y_pred_knn, average=None)
for i in range(4):
    print(f"Класс {i}: Precision={precision[i]:.6f}, Recall={recall[i]:.6f},
F1={f1[i]:.6f}")
```

Результат работы показан на рис. 2.4.



Класс	Precision	Recall	F1
Класс 0	0.952381	1.000000	0.975610
Класс 1	1.000000	1.000000	1.000000
Класс 2	1.000000	1.000000	1.000000
Класс 3	1.000000	0.950000	0.974359

Рисунок 2.4 - Значения Precision, Recall и F1-мера для всех классов.

Классы 1 и 2 имеют идеальные метрики ($\text{Precision} = \text{Recall} = \text{F1} = 1.000$) - все объекты этих классов классифицированы верно. У класса 0 $\text{Recall} = 1.000$ - все объекты найдены, $\text{Precision} = 0.952$ - один объект другого класса ошибочно предсказан как класс 0. У класса 3 $\text{Precision} = 1.000$ - все предсказания верны, $\text{Recall} = 0.950$ - один объект класса 3 пропущен.

Единственная ошибка возникла между классами 0 и 3, что согласуется с таблицей ошибок.

2.5. Построение изображения ROC-кривой, расчет значения AUC для полученных результатов. Вывод о классификации.

ROC-кривая показывает зависимость между True Positive Rate (TPR - количество объектов класса найдено из всех настоящих) и False Positive Rate (FPR - количество объектов других классов, которые ошибочно были приняты за данный класс). AUC - это площадь под ROC-кривой, она показывает способность модели разделять классы.

Построим ROC-кривую для одного из классов и рассчитаем значения AUC для всех классов с помощью функций `roc_curve` и `auc` библиотеки `sklearn`. Выберем классы 3 и 0, так как для них наблюдаются ошибки классификации.

```
from sklearn.metrics import roc_curve, auc
```

```

from sklearn.preprocessing import label_binarize

y_test_bin = label_binarize(y_test, classes=[0,1,2,3])
n_classes = y_test_bin.shape[1]

y_score_knn = knn_best.predict_proba(X_test_scaled)

fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], y_score_knn[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

index_classes = [0, 3]

for ind_class in index_classes:
    plt.plot(fpr[ind_class], tpr[ind_class], label=f'Класс {ind_class} (AUC = {roc_auc[ind_class]:.3f})')
    plt.plot([0, 1], [0, 1], 'k--')
    plt.xlabel('FPR')
    plt.ylabel('TPR')
    plt.title(f'ROC-кривая для {ind_class} класса')
    plt.legend()
    plt.grid()
    plt.show()

for i in range(n_classes):
    print(f"AUC для класса {i}: {roc_auc[i]:.4f}")

mean_auc = np.mean([roc_auc[i] for i in range(n_classes)])
print(f"\nСреднее AUC (macro): {mean_auc:.4f}")

```

Результат работы показан на рис. 2.5.1 и рис. 2.5.2.

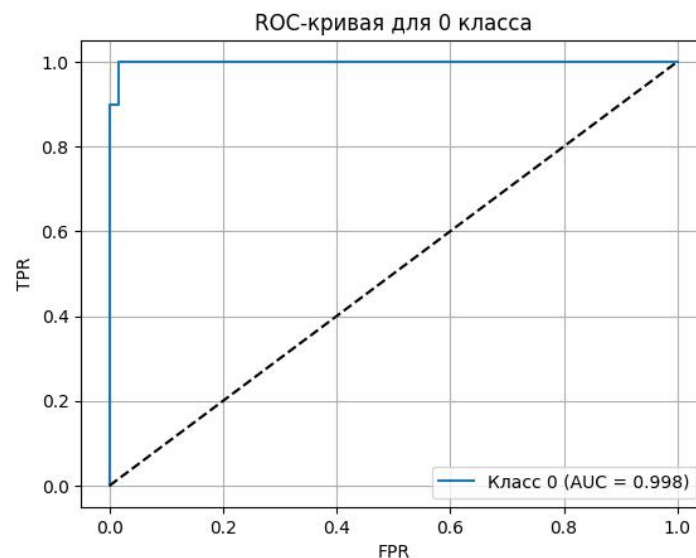


Рисунок 2.5.1 - ROC-кривая kNN для класса 0.

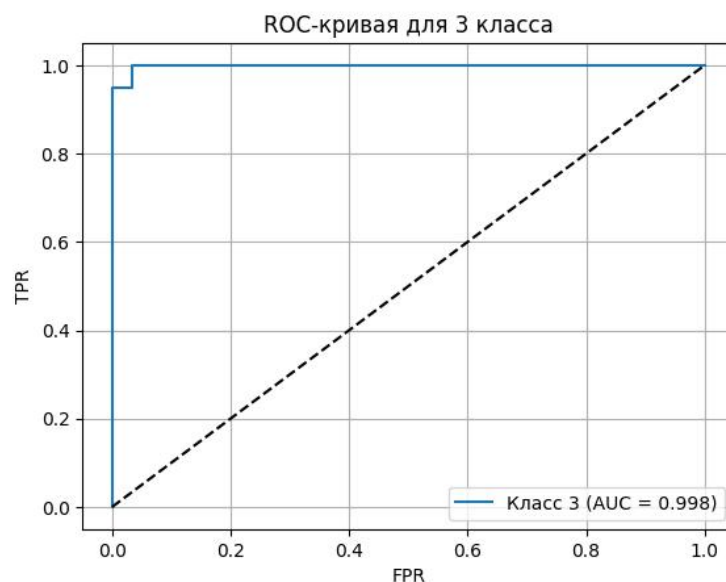


Рисунок 2.5.2 - ROC-кривая kNN для класса 3.

ROC-кривые для классов 0 и 3 расположены выше диагонали случайного классификатора и проходят вблизи верхнего левого угла, что указывает на высокую способность модели kNN отделять целевой класс от остальных. Обе кривые имеют ступенчатую форму, что характерно для kNN, поскольку значения `predict_proba` принимают ограниченное число дискретных уровней и зависят от состава ближайших соседей.

Для классов 1 и 2 значение AUC равно 1.0000, что означает отсутствие ошибок ранжирования. Для классов 0 и 3 значение AUC немного ниже (0.9983), что согласуется с ранее выявленной ошибкой классификации между этими классами. Это указывает на наличие объектов с близкими значениями признаков, для которых модель присваивает схожие вероятности.

3. Логистическая регрессия

3.1. Классификация методом логистической регрессии при различных параметрах (без регуляризации, с l1 регуляризацией, с l2 регуляризацией). Построение диаграммы зависимости точности от наличия регуляризации.

Создадим три модели логистической регрессии без регуляризации, с l1 регуляризацией и с l2 регуляризацией, обучим их и выберем лучшую модель исходя из максимального значения Ассурасу.

```

from sklearn.linear_model import LogisticRegression

logistical_regression_models = {
    'Без регуляризации': LogisticRegression(penalty=None, random_state=42,
max_iter=1000),
    'L1 регуляризация': LogisticRegression(penalty='l1', solver='liblinear',
random_state=42, max_iter=1000),
    'L2 регуляризация': LogisticRegression(penalty='l2', solver='liblinear',
random_state=42, max_iter=1000)
}

accuracies = {}
for name, model in logistical_regression_models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)
    accuracies[name] = acc
    print(f"{name}: Accuracy = {acc:.4f}")

plt.bar(accuracies.keys(), accuracies.values())
plt.xlabel('Тип регуляризации')
plt.ylabel('Accuracy')
plt.title('Точность логистической регрессии при разных типах регуляризации')
for i, (name, acc) in enumerate(accuracies.items()):
    plt.text(i, acc + 0.005, f'{acc:.4f}', ha='center')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

```

Результат работы показан на рис. 3.1.

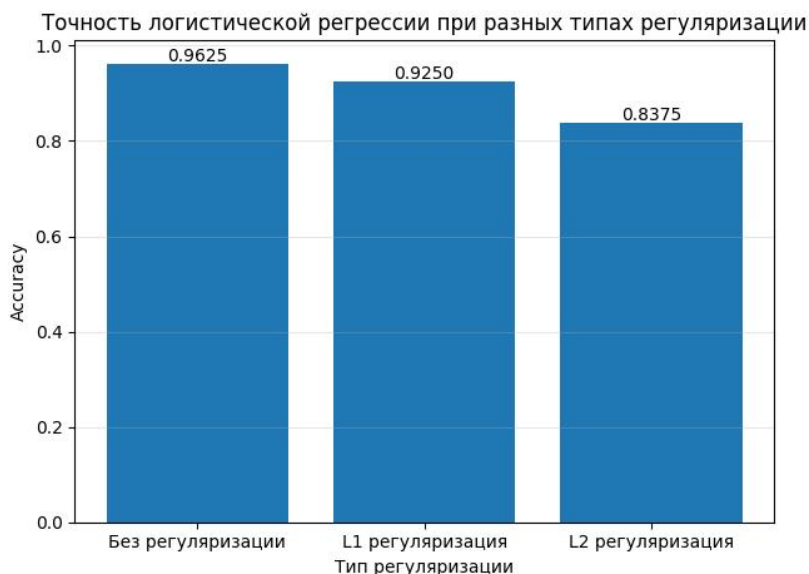


Рисунок 3.1 - Точность логистической регрессии при разных типах регуляризации.

Модель без регуляризации показала результат Accuracy = 0.9625, что является лучшим среди остальных моделей: у модели с L1-регуляризацией

Accuracy = 0.9250, а у модели с L2-регуляризацией худший результат Accuracy = 0.8375.

Регуляризация сильно ограничивает модель, поэтому модели с ней показали себя хуже, чем модель без регуляризации. Набор данных хорошо разделим и не содержит переобучения, поэтому дополнительные ограничения только ухудшают качество классификации.

Для дальнейшей работы выбрана логистическая регрессия без регуляризации (Accuracy = 0.9625), поскольку она показала наилучшую точность на тестовой выборке.

3.2. Построение изображения с границами принятия решения.

Построим границы принятия решений для логистической регрессии без регуляризаций с помощью класса `DecisionBoundaryDisplay` из библиотеки `sklearn`.

```
log_reg_best = LogisticRegression(penalty=None, random_state=42, max_iter=1000)
log_reg_best.fit(X_train_scaled, y_train)

display = DecisionBoundaryDisplay.from_estimator(
    log_reg_best,
    X_scaled,
    response_method='predict',
    xlabel='Experience (масштабированный)',
    ylabel='KPI (масштабированный)',
    grid_resolution=200,
    alpha=0.7,
    cmap='viridis'
)

scatter = plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y, cmap='viridis',
edgecolors='k', s=50, linewidth=0.5)
plt.legend(*scatter.legend_elements(), title="Классы", loc='upper right')
plt.title('Границы принятия решений логистической регрессии')
plt.grid(False)
plt.show()
```

Результат работы показан на рис. 3.2.

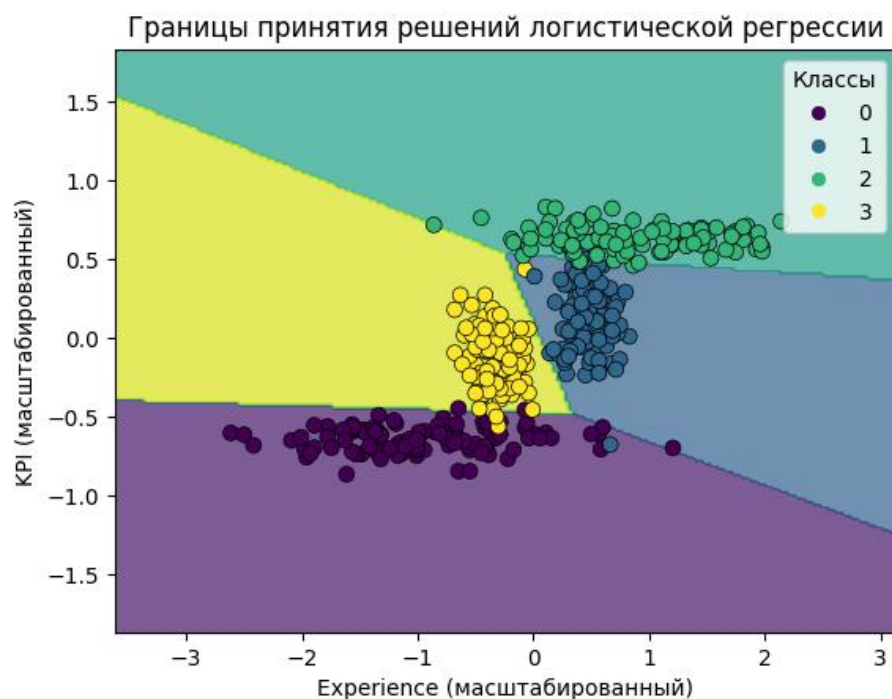


Рисунок 3.2 - Границы принятия решений логистической регрессии.

В отличие от kNN, логистическая регрессия строит линейные (прямолинейные) границы между классами. Метод логистической регрессии ищет линейную комбинацию признаков, которая лучше всего разделяет классы. Границы между классами четкие и прямые. Некоторые классы хорошо разделяются (класс 0 в нижней части), однако классы 1 и 2 имеют зону перекрытия, где линейная граница не может идеально их разделить.

Логистическая регрессия показывает хорошее качество классификации (Ассурасу = 0.9625), но уступает методу kNN (0.9875). Большинство точек находятся в областях своих классов, но отдельные точки попадают в чужие области - это ошибки модели. Линейными границами не всегда можно описать сложные структуры данных.

3.3. Построение таблицы ошибок для полученных результатов. Выводы о классификации на основе таблицы ошибок.

Построим таблицу ошибок логистической регрессии через класс ConfusionMatrixDisplay из библиотеки sklearn.

```
ConfusionMatrixDisplay.from_estimator(
    log_reg_best,
```

```

X_test_scaled,
y_test,
display_labels=['0', '1', '2', '3'],
cmap='Blues',
values_format='d'
)
plt.title('Матрица ошибок логистической регрессии (без регуляризации)')
plt.grid(False)
plt.show()

```

Результат работы показан на рис. 3.3.

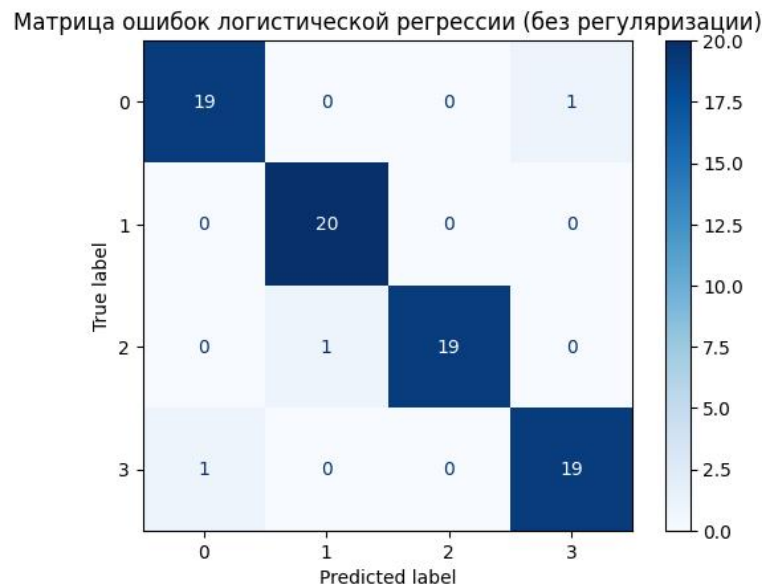


Рисунок 3.3 - Матрица ошибок логистической регрессии.

На матрице ошибок диагональные ячейки показывают количество правильно классифицированных объектов, а остальные - ошибки модели. Из 80 объектов тестовой выборки правильно предсказано 77, что соответствует точности 0.9625.

Ошибки распределены следующим образом: 1 объект класса 0 предсказан как класс 3, 1 объект класса 2 предсказан как класс 1, и 1 объект класса 3 предсказан как класс 0. Это согласуется с визуализацией границ решений: линейные границы не могут идеально разделить классы с нелинейной структурой, особенно в зонах перекрытия (классы 1 и 2).

Класс 1 классифицирован без ошибок (20/20). Остальные классы имеют по 1 ошибке, что указывает на трудности модели в разграничении соседних классов.

3.4. Расчет значений *Precision*, *Recall*, *F1* для полученных результатов.

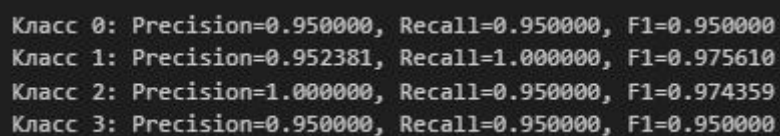
Сопоставление с таблицей ошибок.

Для оценки качества классификации рассчитаем метрики Precision (точность), Recall (полнота) и F1-мера (гармоническое среднее) для каждого класса.

```
y_pred_log_reg = log_reg_best.predict(X_test_scaled)

precision, recall, f1, support = precision_recall_fscore_support(y_test,
y_pred_log_reg, average=None)
for i in range(4):
    print(f"Класс {i}: Precision={precision[i]:.6f}, Recall={recall[i]:.6f},
F1={f1[i]:.6f}")
```

Результат работы показан на рис. 3.4.



Класс	Precision	Recall	F1
Класс 0	0.950000	0.950000	0.950000
Класс 1	0.952381	1.000000	0.975610
Класс 2	1.000000	0.950000	0.974359
Класс 3	0.950000	0.950000	0.950000

Рисунок 3.4 - Значения Precision, Recall и F1-мера для всех классов.

Класс 1 имеет Recall = 1.000 - все объекты этого класса найдены, Precision = 0.952 - один объект другого класса ошибочно предсказан как класс 1. Класс 2 имеет Precision = 1.000 - все предсказания верны, Recall = 0.950 - один объект класса 2 пропущен. Классы 0 и 3 имеют одинаковые метрики (Precision = Recall = F1 = 0.950), что указывает на наличие ошибок в обе стороны: часть объектов пропущена, часть чужих объектов попала в эти классы.

Полученные результаты согласуются с матрицей ошибок из п. 3.3. Ошибки распределены между несколькими классами: объекты классов 0 и 3 частично путаются между собой, а также происходят единичные ошибки между классами 1 и 2.

3.5. Построение изображения ROC-кривой, расчет значения AUC для полученных результатов. Вывод о классификации.

Построим ROC-кривую для одного из классов и рассчитаем значения AUC для всех классов. Выберем классы 0, 2 и 3, поскольку они имеют ошибки в классификации.

```
y_score_log_reg = log_reg_best.predict_proba(X_test_scaled)

fpr = dict()
tpr = dict()
roc_auc = dict()
```

```

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], y_score_log_reg[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

fpr["micro"], tpr["micro"], _ = roc_curve(y_test_bin.ravel(),
y_score_log_reg.ravel())
roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])
roc_auc["macro"] = np.mean([roc_auc[i] for i in range(n_classes)])

ind_classes = [0, 2, 3]

for ind_class in ind_classes:
    plt.plot(fpr[ind_class], tpr[ind_class], label=f'Класс {ind_class} (AUC = {roc_auc[ind_class]:.3f})')
    plt.plot([0, 1], [0, 1], 'k--')
    plt.xlabel('FPR')
    plt.ylabel('TPR')
    plt.title(f'ROC-кривая для {ind_class} класса')
    plt.legend()
    plt.grid()
    plt.show()

for i in range(n_classes):
    print(f"AUC для класса {i}: {roc_auc[i]:.4f}")

```

Результат работы показан на рис. 3.5.1, рис. 3.5.2 и рис. 3.5.3.

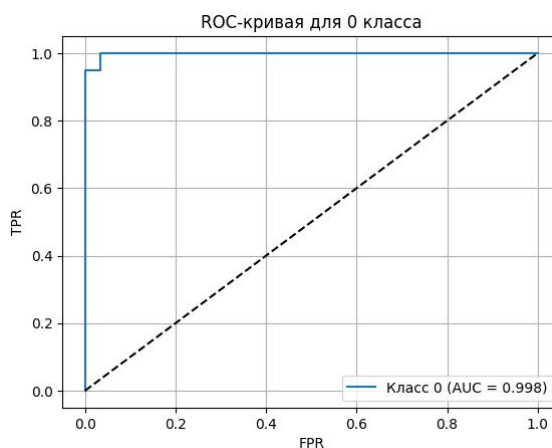


Рисунок 3.5.1 - ROC-кривая логистической регрессии для 0 класса.

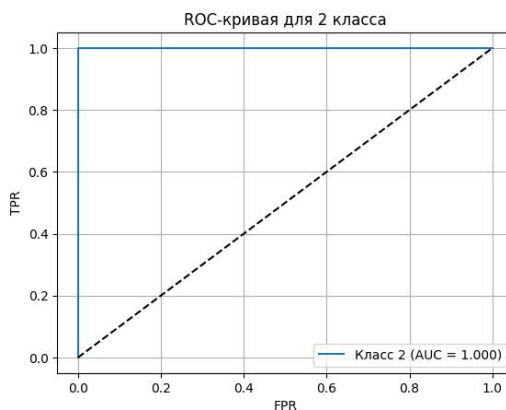


Рисунок 3.5.2 - ROC-кривая логистической регрессии для 2 класса.

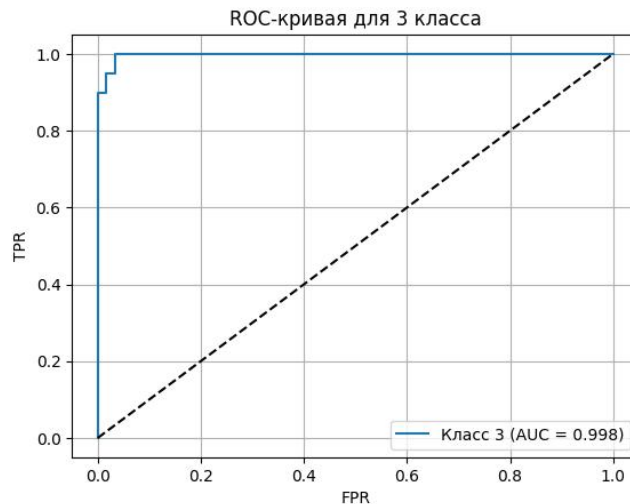


Рисунок 3.5.3 - ROC-кривая логистической регрессии для 3 класса.

ROC-кривые логистической регрессии для классов 0, 2 и 3 расположены также выше диагонали случайного классификатора - это указывает на высокую разделяющую способность модели. Все кривые проходят вблизи верхнего левого угла, поэтому даже при небольшом уровне ошибочно положительных срабатываний достигается высокая полнота обнаружения целевого класса.

Для класса 2 кривая практически совпадает с идеальной, а значение $AUC = 1.000$, что соответствует отсутствию ошибок ранжирования для данного класса. Для классов 0 и 3 значения AUC немного ниже (около 0.998), однако остаются очень высокими, то есть модель в большинстве случаев правильно упорядочивает объекты по вероятности принадлежности к этим классам.

Несмотря на ошибки, значения AUC остаются высокими, так как в большинстве случаев модель сохраняет правильный порядок вероятностей: объекты своего класса получают более высокие оценки, чем объекты других классов.

4. Метод опорных векторов

4.1. Классификация методом опорных векторов при различных параметрах (*linear*, *poly* (нужно выбрать степень), *rbf*). Построение диаграммы зависимости точности от вида ядра.

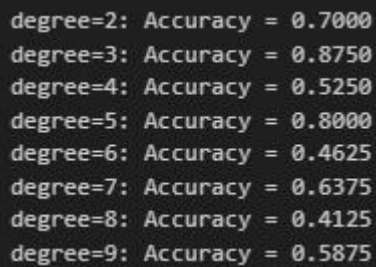
Выберем наилучшую степень для опорного вектора с ядром `poly` с помощью класса `SVC` библиотеки `sklearn`.

```
from sklearn.svm import SVC

poly_accuracies = []

for i in range (2, 10):
    model = SVC(kernel='poly', degree=i, probability=True)
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)
    poly_accuracies.append(acc)
    print(f"degree={i}: Accuracy = {acc:.4f}")
```

Результат работы показан на рис. 4.1.1.



degree	Accuracy
2	0.7000
3	0.8750
4	0.5250
5	0.8000
6	0.4625
7	0.6375
8	0.4125
9	0.5875

Рисунок 4.1.1 - Accuracy степеней ядра `poly`.

Наилучшее значение Accuracy было достигнуто у модели со степенью 3, поэтому в дальнейшем сравнении ядер опорных векторов будем рассматривать `poly` со степенью 3.

Создадим три модели опорных векторов с ядрами `linear`, `poly` (степень 3) и `rbf`, обучим их и выберем лучшую модель исходя из максимального значения Accuracy.

```
kernels = ['linear', 'poly', 'rbf']
accuracies = []

for kernel in kernels:
    if kernel == 'poly':
        model = SVC(kernel=kernel, degree=3, probability=True)
    else:
        model = SVC(kernel=kernel, probability=True)

    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)
    accuracies.append(acc)
    print(f"{kernel}: Accuracy = {acc:.4f}")

plt.bar(kernels, accuracies)
plt.ylabel('Accuracy')
plt.title('Зависимость Accuracy от ядра SVM')
```

```
for i, v in enumerate(accuracies):
    plt.text(i, v + 0.001, f"{v:.3f}", ha='center')
plt.grid(axis='y', alpha=0.3)
plt.show()
```

Результат работы показан на рис. 4.1.2.

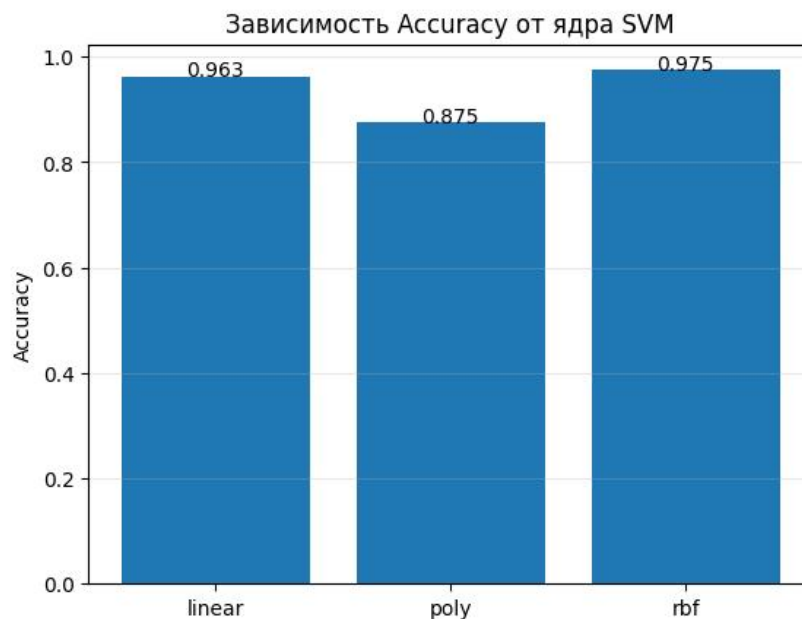


Рисунок 4.1.2 - Точность опорных векторов при разных ядрах.

Модель с ядром rbf показала наибольшее значение Accuracy = 0.9750, что выше, чем у моделей с ядрами linear (0.9625) и poly (0.8750).

Более высокое качество модели с ядром rbf объясняется тем, что данное ядро позволяет строить нелинейные границы решений и лучше учитывать структуру данных, особенно в областях, где классы частично перекрываются. Модель с линейным ядром показывает немного более низкое качество, что связано с ограничением в виде линейной границы, которая не может полностью разделить классы. Полиномиальное ядро со степенью 3 демонстрирует наименьшее значение Accuracy - это указывает на избыточную сложность модели и ухудшение обобщающей способности.

Для дальнейшей работы выбрана модель с ядром rbf, поскольку она показала наилучшую точность на тестовой выборке.

4.2. Построение изображения с границами принятия решения.

Построим границы принятия решений для опорных векторов с помощью класса `DecisionBoundaryDisplay` из библиотеки `sklearn`.

```
svc_best = SVC(kernel='rbf', probability=True)
svc_best.fit(X_train_scaled, y_train)

display = DecisionBoundaryDisplay.from_estimator(
    svc_best,
    X_scaled,
    response_method='predict',
    xlabel='Experience (масштабированный)',
    ylabel='KPI (масштабированный)',
    grid_resolution=200,
    alpha=0.7,
    cmap='viridis'
)

scatter = plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y, cmap='viridis',
                     edgecolors='k', s=50, linewidth=0.5)
plt.legend(*scatter.legend_elements(), title="Классы", loc='upper right')
plt.title('Границы принятия решений опорных векторов')
plt.grid(False)
plt.show()
```

Результат работы показан на рис. 4.2.

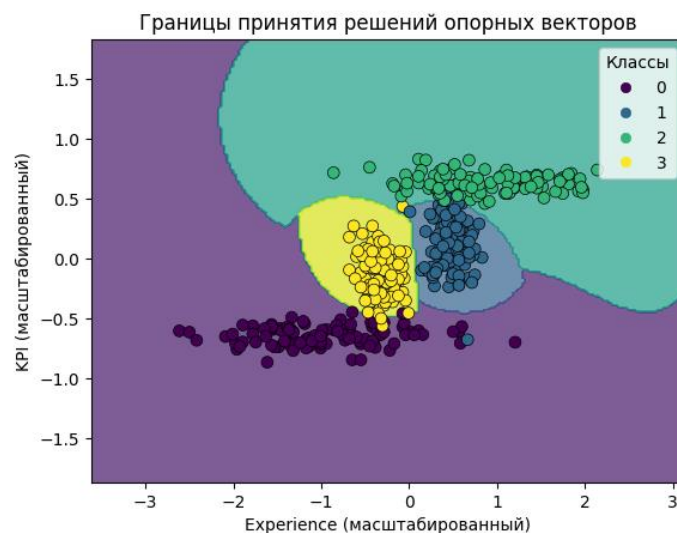


Рисунок 4.2 - Границы принятия решений опорных векторов.

Границы принятия решений метода опорных векторов с ядром `rbf` имеют нелинейный характер и представляют собой гладкие кривые, адаптирующиеся к распределению данных. В отличие от линейной модели, SVM с `rbf` способен учитывать сложную структуру данных и формировать замкнутые области для отдельных классов.

Класс 0 выделен в нижней части пространства признаков, класс 3 - в центральной области. Классы 1 и 2 расположены близко друг к другу, однако модель формирует между ними изогнутую границу, позволяющую лучше разделить объекты по сравнению с линейным подходом.

Большинство точек находятся внутри соответствующих областей своих классов, что согласуется с высоким значением Accuracy = 0.9750.

4.3. Построение таблицы ошибок для полученных результатов. Выводы о классификации на основе таблицы ошибок.

Построим таблицу ошибок опорных векторов через класс ConfusionMatrixDisplay из библиотеки sklearn.

```
y_pred_svc = svc_best.predict(X_test_scaled)

ConfusionMatrixDisplay.from_estimator(
    svc_best,
    X_test_scaled,
    y_test,
    display_labels=['0', '1', '2', '3'],
    cmap='Blues',
    values_format='d'
)
plt.title('Матрица ошибок опорных векторов с ядром rbf')
plt.grid(False)
plt.show()
```

Результат работы показан на рис. 4.3.

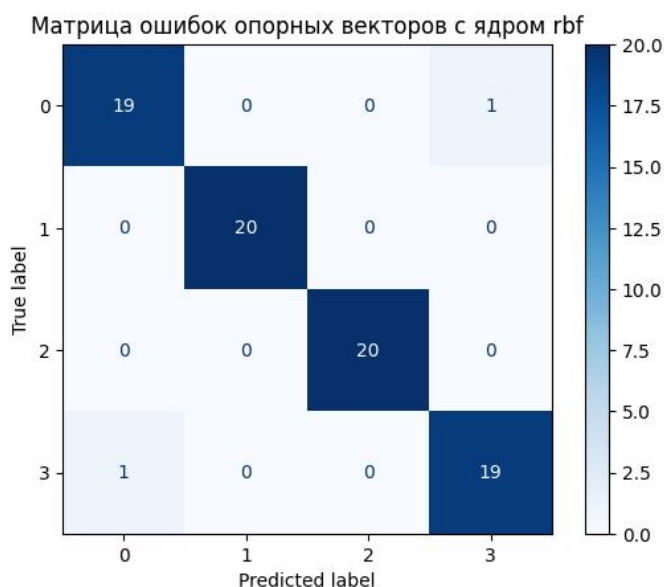


Рисунок 4.3 - Матрица ошибок опорных векторов.

Из 80 объектов тестовой выборки правильно предсказано 78, что соответствует точности 0.9750.

Один объект класса 0 отнесен к классу 3 и один объект класса 3 отнесен к классу 0. Таким образом, ошибки наблюдаются только между классами 0 и 3. Для классов 1 и 2 ошибок нет - все объекты этих классов классифицированы верно (20 из 20).

Это согласуется с ранее полученными результатами: классы 1 и 2 хорошо разделяются моделью, тогда как классы 0 и 3 имеют близкие значения признаков в отдельных областях пространства, что приводит к единичным ошибкам на границе между ними.

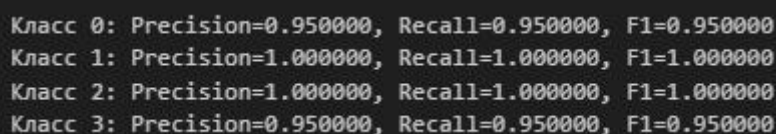
4.4. Расчет значений Precision, Recall, F1 для полученных результатов. Сопоставление с таблицей ошибок.

Для оценки качества классификации рассчитаем метрики Precision (точность), Recall (полнота) и F1-мера (гармоническое среднее) для каждого класса.

```
from sklearn.metrics import precision_recall_fscore_support

precision, recall, f1, support = precision_recall_fscore_support(y_test,
y_pred_svc, average=None)
for i in range(4):
    print(f"Класс {i}: Precision={precision[i]:.6f}, Recall={recall[i]:.6f},
F1={f1[i]:.6f}")
```

Результат работы показан на рис. 4.4.



Класс	Precision	Recall	F1
Класс 0	0.950000	0.950000	0.950000
Класс 1	1.000000	1.000000	1.000000
Класс 2	1.000000	1.000000	1.000000
Класс 3	0.950000	0.950000	0.950000

Рисунок 4.4 - Значения Precision, Recall и F1-мера для всех классов.

Классы 1 и 2 имеют идеальные метрики ($\text{Precision} = \text{Recall} = \text{F1} = 1.000$), что означает, что все объекты этих классов классифицированы верно и не было ни ложных срабатываний, ни пропусков.

Классы 0 и 3 имеют одинаковые значения метрик ($\text{Precision} = \text{Recall} = \text{F1} = 0.950$). Это указывает на наличие ошибок между этими классами: один объект

класса 0 был отнесен к классу 3, и один объект класса 3 - к классу 0. В результате для каждого из этих классов наблюдается как один пропущенный объект, так и одно ложное срабатывание.

Полученные значения полностью согласуются с матрицей ошибок из п. 4.3: ошибки возникают только между классами 0 и 3, тогда как классы 1 и 2 разделяются без ошибок.

4.5. Построение изображения ROC-кривой, расчет значения AUC для полученных результатов. Вывод о классификации.

Построим ROC-кривую для одного из классов и рассчитаем значения AUC для всех классов. Выберем классы 3 и 0, так как для них наблюдаются ошибки классификации.

```
y_score_svc = svc_best.predict_proba(X_test_scaled)

fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], y_score_svc[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

fpr["micro"], tpr["micro"], _ = roc_curve(y_test_bin.ravel(),
y_score_svc.ravel())
roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])
roc_auc["macro"] = np.mean([roc_auc[i] for i in range(n_classes)])

ind_classes = [0, 3]

for ind_class in ind_classes:
    plt.plot(fpr[ind_class], tpr[ind_class], label=f'Класс {ind_class} (AUC =
{roc_auc[ind_class]:.3f})')
    plt.plot([0, 1], [0, 1], 'k--')
    plt.xlabel('FPR')
    plt.ylabel('TPR')
    plt.title(f'ROC-кривая для {ind_class} класса')
    plt.legend()
    plt.grid()
    plt.show()

for i in range(n_classes):
    print(f"AUC для класса {i}: {roc_auc[i]:.4f}")
```

Результат работы показан на рис. 4.5.1 и рис. 4.5.2.

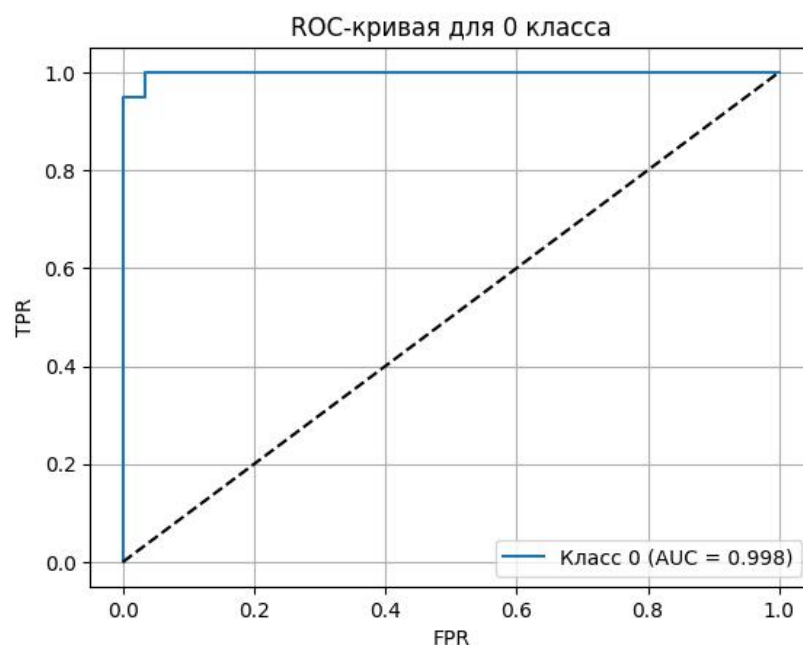


Рисунок 4.5.1 - ROC-кривая опорных векторов для 0 класса.

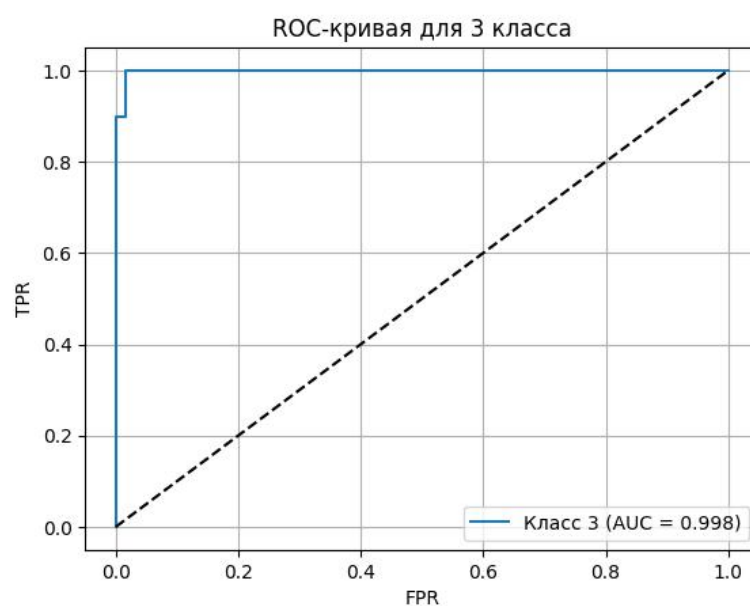


Рисунок 4.5.2 - ROC-кривая опорных векторов для 3 класса.

ROC-кривая для класса 3 располагается выше диагонали случайного классификатора и проходит близко к верхней границе графика. Кривая имеет ступенчатый вид с одним выраженным переходом - это связано с изменением значений вероятностей для объектов, находящихся вблизи границы между классами.

Значения AUC для классов 0 и 3 составляют примерно 0.998, то есть модель почти идеально ранжирует объекты этих классов. Для классов 1 и 2 значение AUC = 1.0000, что соответствует полностью корректному разделению без ошибок ранжирования.

Результаты согласуются с матрицей ошибок: основные трудности возникают только при различении классов 0 и 3, тогда как классы 1 и 2 разделяются уверенно.

5. Решающие деревья

5.1. Классификация методом решающих деревьев при различных параметрах (максимальная глубина, максимальное количество листьев, метрика загрязнения). Построение диаграммы зависимости точности от разных параметров.

Проведем подбор гиперпараметров модели решающего дерева, чтобы получить наилучшее обобщение. Рассмотрим следующие параметры: `max_depth` (максимальная глубина дерева), `max_leaf_nodes` (максимальное количество листьев), `criterion` (метрика загрязнения (`gini`, `entropy`, `log_loss`)).

Подберем `max_depth`:

```
from sklearn.tree import DecisionTreeClassifier

depths = range(1, 21)
depth_accuracies = []

for d in depths:
    model = DecisionTreeClassifier(max_depth=d, random_state=42)
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)
    depth_accuracies.append(acc)
    print(f"max_depth={d}: Accuracy = {acc:.4f}")

best_depth = depths[depth_accuracies.index(max(depth_accuracies))]
print(f"\nЛучшая глубина: max_depth={best_depth}, Accuracy={max(depth_accuracies):.4f}")

plt.plot(depths, depth_accuracies, marker='o')
plt.xlabel('max_depth')
plt.ylabel('Accuracy')
plt.title('Зависимость Accuracy от max_depth (решающие деревья)')
plt.grid(alpha=0.3)
plt.show()
```

Результат работы показан на рис. 5.1.1.

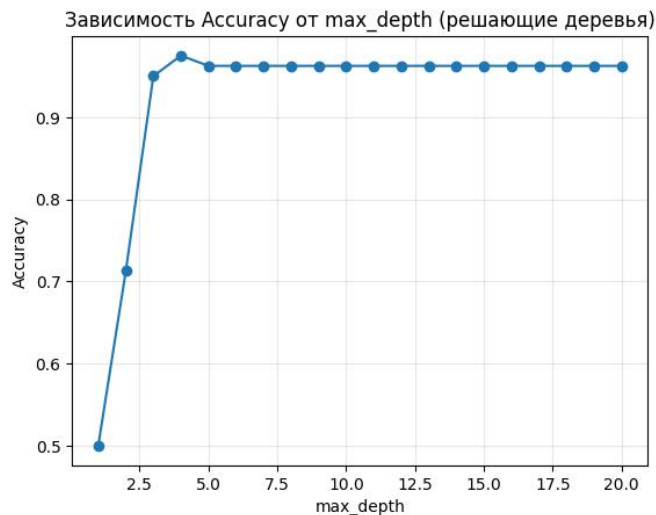


Рисунок 5.1.1 - Зависимость Accurasy от max_depth.

При малой глубине дерева видно, что модель недообучается (точность 0.50 - 0.70), т. к. дерево слишком простое и не улавливает структуру данных. Начиная с глубины 3-4 наблюдается резкий рост качества (до 0.95-0.975) - модель начинает выделять информативные правила разделения классов.

Максимальная точность на тестовой выборке достигается при max_depth = 4 (Accurasy = 0.9750), будем использовать это значение для параметра. Не берем значение больше, поскольку усложнения дерева после глубины 4 не дает прирост качества, а только увеличивает риск переобучения.

Подберем max_leaf_nodes:

```
leaf_nodes = range(2, 21)
leaf_accuracies = []

for n in leaf_nodes:
    model = DecisionTreeClassifier(max_depth=best_depth, max_leaf_nodes=n,
random_state=42)
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)
    leaf_accuracies.append(acc)
    print(f"max_leaf_nodes={n}: Accuracy = {acc:.4f}")

best_leaf_nodes = leaf_nodes[leaf_accuracies.index(max(leaf_accuracies))]
print(f"\nЛучшее количество листьев: max_leaf_nodes={best_leaf_nodes},
Accuracy={max(leaf_accuracies):.4f}")

plt.plot(leaf_nodes, leaf_accuracies, marker='o')
plt.xlabel('max_leaf_nodes')
plt.ylabel('Accuracy')
plt.title('Зависимость Accurasy от max_leaf_nodes (решающие деревья)')
plt.grid(alpha=0.3)
```

```
plt.show()
```

Результат работы показан на рис. 5.1.2.

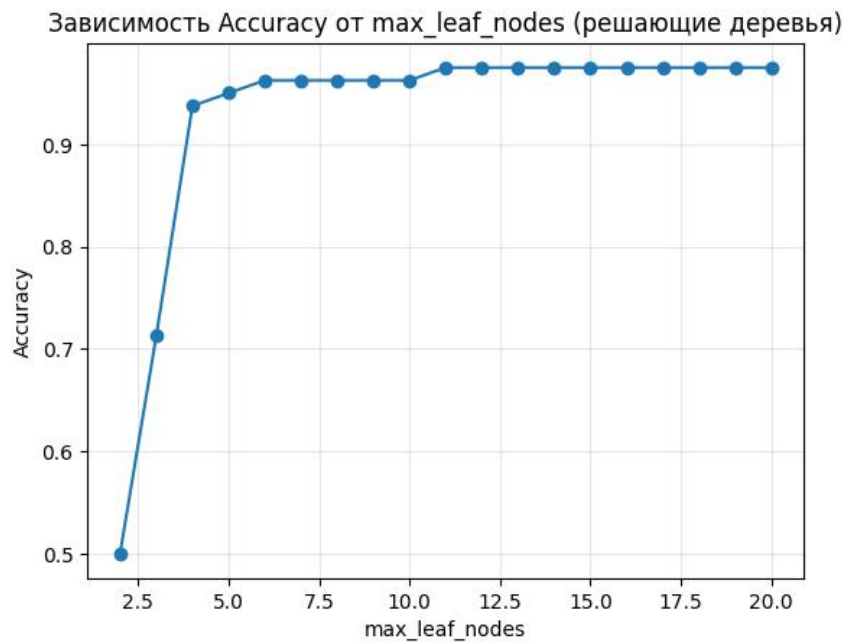


Рисунок 5.1.2 - Зависимость Accuracy от max_leaf_nodes.

Аналогично при подборе max_leaf_nodes лучшим оказалось значение max_leaf_nodes = 11 (Accuracy = 0.9750): меньшее число листьев ограничивает выразительность модели, большее - не дает дополнительного выигрыша в обобщающей способности.

Подберем criterion:

```
criteria = ['gini', 'entropy', 'log_loss']
criterion_accuracies = []

for c in criteria:
    model = DecisionTreeClassifier(
        criterion=c,
        max_depth=best_depth,
        max_leaf_nodes=best_leaf_nodes,
        random_state=42
    )
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)
    criterion_accuracies.append(acc)
    print(f"{c}: Accuracy = {acc:.4f}")

best_criterion = criteria[criterion_accuracies.index(max(criterion_accuracies))]
print(f"\nЛучший критерий: {best_criterion}, Accuracy={max(criterion_accuracies):.4f}")

plt.bar(criteria, criterion_accuracies)
```

```
plt.ylabel('Accuracy')
plt.title('Зависимость Accuracy от критерия загрязнения')
for i, v in enumerate(criterion_accuracies):
    plt.text(i, v + 0.001, f"{v:.3f}", ha='center')
plt.grid(axis='y', alpha=0.3)
plt.show()
```

Результат работы показан на рис. 5.1.3.



Рисунок 5.1.3 - Зависимость Accuracy от criterion.

Максимальная точность на тестовой выборке достигается при criterion = 'gini' (Accuracy = 0.9750), будем использовать это значение для параметра.

Построим модель решающих деревьев с параметрами, которые дали наилучшую точность : max_depth = 4, max_leaf_nodes = 11, criterion = 'gini'.

```
decision_three_best = DecisionTreeClassifier(
    criterion=best_criterion,
    max_depth=best_depth,
    max_leaf_nodes=best_leaf_nodes,
    random_state=42
)
decision_three_best.fit(X_train_scaled, y_train)
y_pred_decision_three = decision_three_best.predict(X_test_scaled)
best_acc = accuracy_score(y_test, y_pred_decision_three)

print(f"Accuracy = {best_acc:.4f}")
```

Результат работы показан на рис. 5.1.4.

Accuracy = 0.9750

Рисунок 5.1.4 - Accuracy модели решающих деревьев.

Итоговая модель показала $\text{Assurasy} = 0.9750$ что подтверждает корректность выбранных гиперпараметров и хорошую обобщающую способность модели.

5.2. Построение изображения с границами принятия решения.

Построим границы принятия решений для решающего дерева с помощью класса `DecisionBoundaryDisplay` из библиотеки `sklearn`.

```
display = DecisionBoundaryDisplay.from_estimator(
    decision_three_best,
    X_scaled,
    response_method='predict',
    xlabel='Experience (масштабированный)',
    ylabel='KPI (масштабированный)',
    grid_resolution=200,
    alpha=0.7,
    cmap='viridis'
)

scatter = plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y, cmap='viridis',
                    edgecolors='k', s=50, linewidth=0.5)
plt.legend(*scatter.legend_elements(), title="Классы", loc='upper right')
plt.title('Границы принятия решений решающего дерева')
plt.grid(False)
plt.show()
```

Результат работы показан на рис. 5.2.

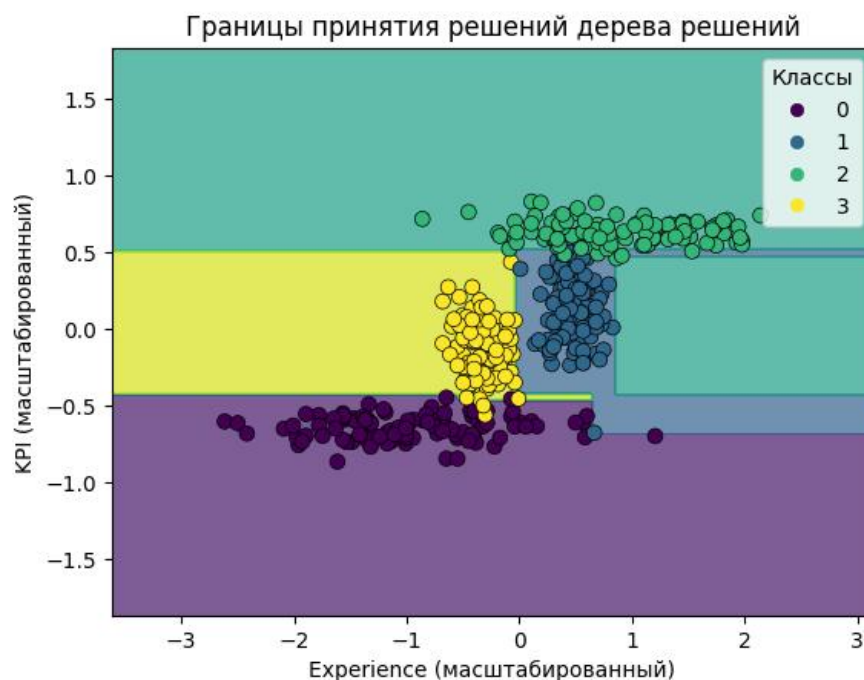


Рисунок 5.2 - Границы принятия решений дерева решений.

Границы принятия решений модели решающего дерева имеют ступенчатый (прямоугольный) вид, так как дерево выполняет последовательные пороговые разбиения признаков.

По распределению областей видно, что модель успешно отделяет класс 0 в нижней части пространства признаков при меньших значениях KPI, а класс 2 - в верхней области при больших значениях KPI. В центральной части сформированы отдельные области для классов 1 и 3, которые расположены ближе друг к другу и потому разделяются сложнее. Несмотря на частичное пересечение точек вблизи границ, большая часть объектов попадает в соответствующие области своих классов.

Большинство точек находятся внутри соответствующих областей своих классов, что согласуется с высоким значением Accuracy = 0.9750.

5.3. Построение таблицы ошибок для полученных результатов. Выводы о классификации на основе таблицы ошибок.

Построим таблицу ошибок решающего дерева через класс ConfusionMatrixDisplay из библиотеки sklearn.

```
ConfusionMatrixDisplay.from_estimator(  
    decision_three_best,  
    X_test_scaled,  
    y_test,  
    display_labels=['0', '1', '2', '3'],  
    cmap='Blues',  
    values_format='d'  
)  
plt.title('Матрица ошибок решающего дерева')  
plt.grid(False)  
plt.show()
```

Результат работы показан на рис. 5.3.

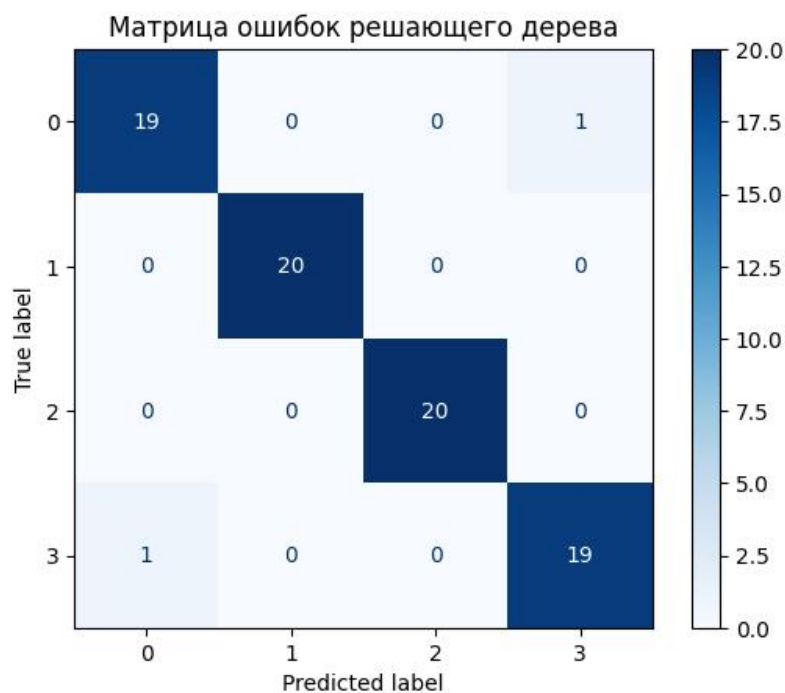


Рисунок 5.3 - Матрица ошибок решающего дерева.

Из 80 объектов тестовой выборки правильно предсказано 78, что соответствует точности 0.9750.

Это согласуется с ранее полученными результатами: классы 1 и 2 хорошо разделяются моделью, тогда как классы 0 и 3 имеют близкие значения признаков в отдельных областях пространства, что приводит к единичным ошибкам на границе между ними.

5.4. Расчет значений Precision, Recall, F1 для полученных результатов. Сопоставление с таблицей ошибок.

Для оценки качества классификации рассчитаем метрики Precision (точность), Recall (полнота) и F1-мера (гармоническое среднее) для каждого класса.

```
precision, recall, f1, support = precision_recall_fscore_support(y_test,
y_pred_decision_three, average=None)
for i in range(4):
    print(f"Класс {i}: Precision={precision[i]:.6f}, Recall={recall[i]:.6f},
F1={f1[i]:.6f}")
```

Результат работы показан на рис. 5.4.

```
Класс 0: Precision=0.950000, Recall=0.950000, F1=0.950000
Класс 1: Precision=1.000000, Recall=1.000000, F1=1.000000
Класс 2: Precision=1.000000, Recall=1.000000, F1=1.000000
Класс 3: Precision=0.950000, Recall=0.950000, F1=0.950000
```

Рисунок 5.4 - Значения Precision, Recall и F1-мера для всех классов.

Классы 1 и 2 распознаются безошибочно: для них Precision = 1.000, Recall = 1.000, F1 = 1.000. Это означает, что модель не допускает ни ложных срабатываний, ни пропусков для данных классов.

Для классов 0 и 3 получены одинаковые значения Precision = 0.950, Recall = 0.950, F1 = 0.950. Снижение метрик связано с взаимной путаницей этих двух классов: часть объектов класса 0 определяется как класс 3, и аналогично часть объектов класса 3 определяется как класс 0.

Результаты по метрикам полностью согласуются с матрицей ошибок из п. 5.3: модель в целом демонстрирует высокое качество классификации, а ошибки локализованы только в паре классов 0 и 3.

5.5. Построение изображения ROC-кривой, расчет значения AUC для полученных результатов. Вывод о классификации.

Построим ROC-кривую для одного из классов и рассчитаем значения AUC для всех классов. Выберем классы 3 и 0, так как для них наблюдаются ошибки классификации.

```
y_score_svc = decision_three_best.predict_proba(X_test_scaled)

fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], y_score_svc[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

fpr["micro"], tpr["micro"], _ = roc_curve(y_test_bin.ravel(),
y_score_svc.ravel())
roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])
roc_auc["macro"] = np.mean([roc_auc[i] for i in range(n_classes)])

ind_classes = [0, 3]

for ind_class in ind_classes:
    plt.plot(fpr[ind_class], tpr[ind_class], label=f'Класс {ind_class} (AUC =
{roc_auc[ind_class]:.3f})')
```

```
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('FPR')
plt.ylabel('TPR')
plt.title(f'ROC-кривая для {ind_class} класса')
plt.legend()
plt.grid()
plt.show()

for i in range(n_classes):
    print(f"AUC для класса {i}: {roc_auc[i]:.4f}")
```

Результат работы показан на рис. 5.5.1 и рис. 5.5.2.

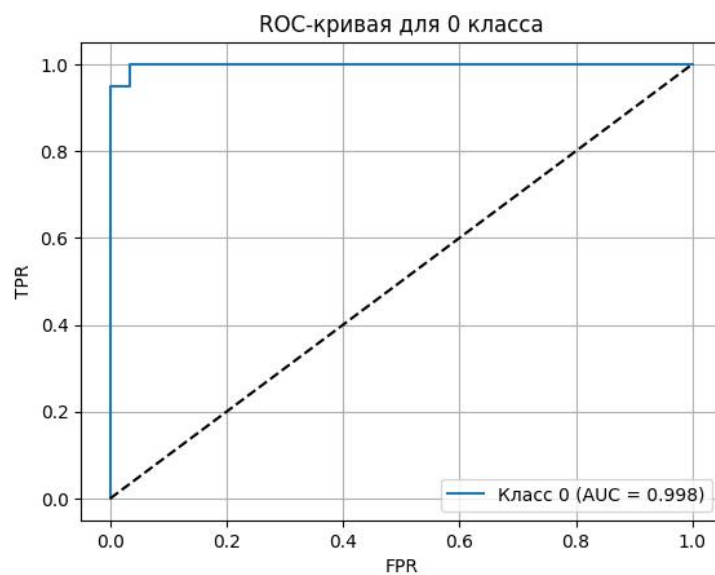


Рисунок 5.5.1 - ROC-кривая решающего дерева для 0 класса.

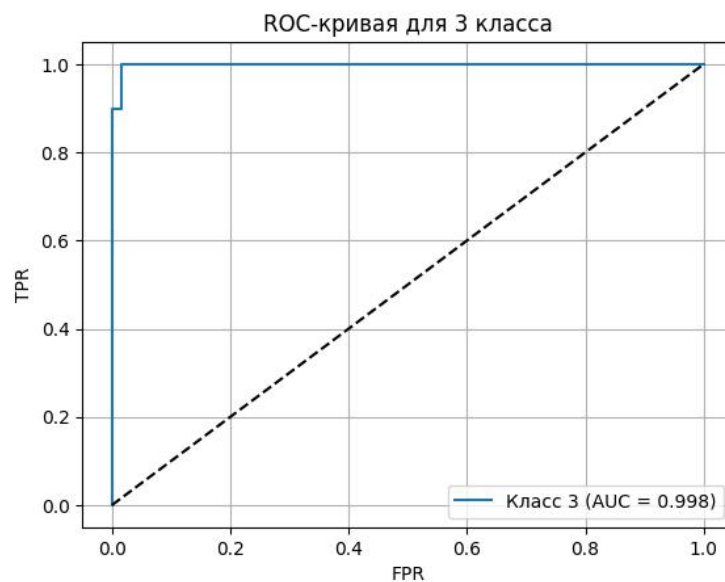


Рисунок 5.5.2 - ROC-кривая решающего дерева для 3 класса.

ROC-кривые для классов 0 и 3 располагаются выше диагонали случайного классификатора и проходят близко к верхней границе графика. Кривые имеют ступенчатый вид с одним выраженным переходом - это связано с дискретным характером выдаваемых вероятностей.

Рассчитанные значения AUC подтверждают высокую точность классификации: для классов 1 и 2 $AUC = 1.0000$, для классов 0 и 3 $AUC = 0.9983$. Это полностью согласуется с матрицей ошибок и метриками Precision/Recall/F1: ошибки возникают только между классами 0 и 3, тогда как классы 1 и 2 разделяются безошибочно.

Результаты по ROC-AUC получились одинаковыми с SVC, что показывает, что обе модели на данном наборе данных обеспечивают одинаково высокую классификацию.

5.6. Визуализация полученного дерева решений. Выводы о правилах в узлах. Сопоставление с границами принятия решений.

Визуализируем полученное дерево решений через функцию `plot_tree` из библиотеки `sklearn`.

```
from sklearn.tree import plot_tree

plt.figure(figsize=(20, 10))
plot_tree(
    decision_three_best,
    feature_names=['Experience', 'KPI'],
    class_names=['0', '1', '2', '3'],
    filled=True,
    rounded=True,
    fontsize=10
)
plt.title('Визуализация решающего дерева')
plt.show()
```

Результат работы показан на рис. 5.6.

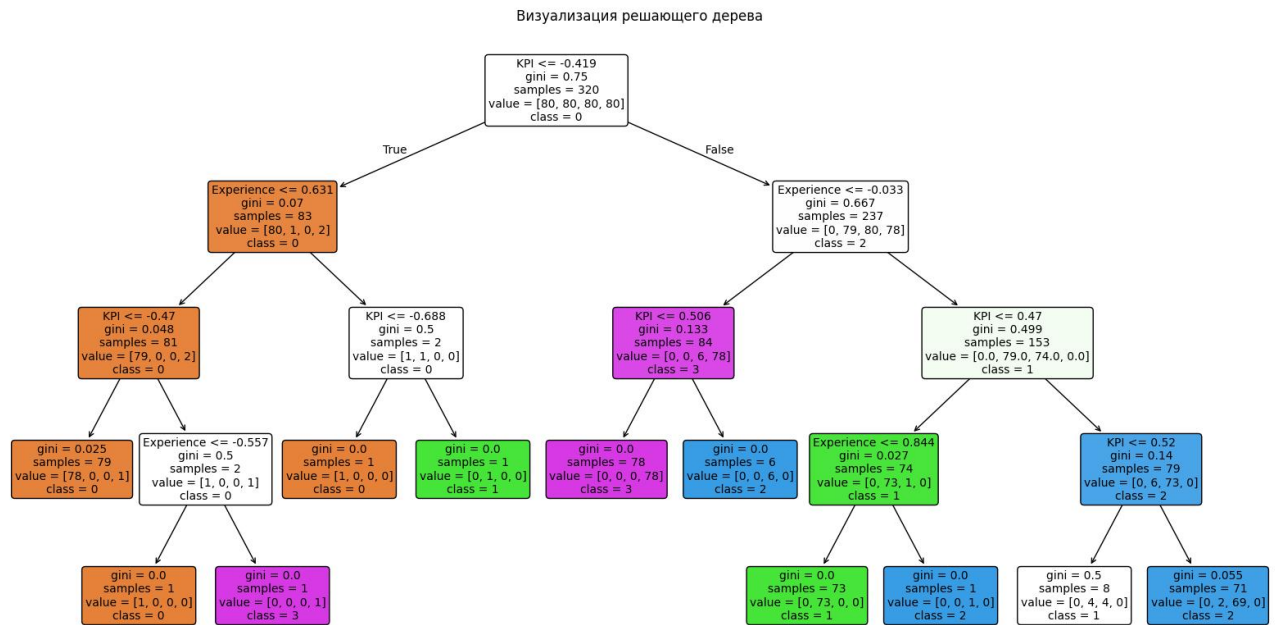


Рисунок 5.6 - Дерево решений.

В корневом узле используется условие $KPI \leq -0.419$, то есть первичное разделение выполняется по признаку KPI. Левая ветвь (при меньших значениях KPI) в основном соответствует классу 0, что подтверждается высокой долей объектов этого класса в узлах и низкими значениями gini. Правая ветвь далее разделяется по признаку Experience ($Experience \leq -0.033$) и дополнительным порогам KPI, формируя области для классов 1, 2 и 3.

По структуре дерева видно, что классы 1 и 2 отделяются наиболее уверенно: для них присутствуют крупные листовые узлы с gini, близким к нулю, и практически однородным составом объектов. Класс 3 также выделяется отдельной областью (ветвь с условием $KPI \leq 0.506$), где большинство объектов относится именно к классу 3. Небольшое смещение наблюдается в отдельных промежуточных узлах между классами 0 и 3, что согласуется с ранее полученной матрицей ошибок, где ошибки возникают только между этими двумя классами.

Полученные правила в узлах сопоставимы с границами принятия решений из п. 5.2: пороговые условия по двум признакам формируют на плоскости ступенчатые прямоугольные области.

6. Выбор классификатора

6.1. Построение таблицы с метриками *Precision*, *Recall*, *AUC* для полученных результатов каждым классификатором.

Построим таблицу с усредненными по классам метриками разных классификаторов для полученных результатов (см. Таб. 1)

Таблица 1 - *Precision*, *Recall*, *AUC* классификаторов для полученных результатов.

Классификатор	<i>Precision</i>	<i>Recall</i>	<i>AUC</i>
kNN	0.9881	0.9875	0.9992
Логистическая регрессия	0.9631	0.9625	0.9990
Опорные вектора	0.9750	0.9750	0.9992
Решающие деревья	0.9750	0.9750	0.9992

Выводы о Таб. 1 будут приведены в п. 6.2.

6.2. Выводы о классификаторах.

По результатам таблицы 1 все рассмотренные классификаторы показывают высокое качество классификации, так как значения *AUC* для всех моделей близки к 1.000. Каждая модель в целом хорошо разделяет классы и корректно ранжирует объекты по вероятности принадлежности.

Наилучшие значения *Precision* и *Recall* получены у kNN (0.9881 и 0.9875) - это делает его наиболее точным классификатором в рамках проведенного сравнения. Логистическая регрессия показывает более низкие значения (0.9631 и 0.9625), и уступает остальным моделям по качеству итоговой классификации. Метод опорных векторов и решающее дерево показали одинаковые результаты (*Precision* = *Recall* = 0.9750, *AUC* = 0.9992), обеспечив высокий и стабильный уровень качества.

Таким образом, по совокупности метрик целесообразно выбрать kNN как основной классификатор для данной задачи, поскольку он обеспечивает наилучший баланс точности и полноты при одинаково высоком уровне AUC.

Ссылка на полный код находится в Приложении.

Выводы

В ходе работы были исследованы и сравнены несколько методов многоклассовой классификации: kNN, логистическая регрессия, метод опорных векторов и решающие деревья. Для каждой модели выполнен подбор гиперпараметров, построены границы принятия решений, матрицы ошибок, рассчитаны метрики Precision, Recall, F1, а также ROC-кривые и значения AUC.

Полученные результаты показали, что все рассмотренные классификаторы обеспечивают высокое качество распознавания классов. Значения AUC для всех моделей близки к 1, что указывает на хорошую разделяющую способность и корректное ранжирование объектов по вероятностям классов. По усредненным метрикам наилучший результат продемонстрировал kNN (Precision = 0.9881, Recall = 0.9875, AUC = 0.9992). Метод опорных векторов и решающее дерево показали сопоставимое качество (Precision = Recall = 0.9750, AUC = 0.9992), а логистическая регрессия дала немного более низкие значения (Precision = 0.9631, Recall = 0.9625, AUC = 0.9990).

Анализ матриц ошибок и поклассовых метрик показал, что основные трудности классификации связаны с близкими по признаковому пространству классами 0 и 3, тогда как классы 1 и 2 разделяются практически безошибочно. Визуализация границ принятия решений подтвердила особенности каждого метода: kNN и SVM формируют более гибкие нелинейные границы, а дерево решений - интерпретируемые пороговые разбиения.

Таким образом, по совокупности метрик и качеству обобщения в данной работе целесообразно выбрать классификатор kNN как наиболее точный. При этом SVM и решающее дерево также могут рассматриваться как эффективные альтернативы с высоким уровнем качества классификации.

ПРИЛОЖЕНИЕ

1. Ссылка на код:

<https://colab.research.google.com/drive/1HvPrDSZ8yFPKnKSFEIbzhQsWkdcw-XCG?usp=sharing>.